DSAI 352	Computer Vision	Final Project

Tin Can Alley

Automated Detection, Tracking & Fall Analysis

Omar Zayed	Karim Wael
Student ID: 202300785	Student ID: 202202212

mAP@50	Cans Detected	Falls Confirmed	Training Epochs	Video Length
99.5%	7	6 / 6	50	30.1 s

Deliverable: Jupyter Notebook: `Cv (1).ipynb` — Colab workflow: upload video + YOLOv8 dataset ZIP, run cells top-to-bottom.

1. Project Overview

We built an **end-to-end computer vision pipeline** for a mobile-recorded "tin can alley" style game video. The system detects soda cans, tracks each can across frames, decides when a can has fallen using an operational rule on bounding-box shape, and exports a new MP4 with overlays (per-can fall timestamp + end summary).

To satisfy the course **modeling requirement**, we did NOT use a pre-trained detector as-is. We **fine-tuned YOLOv8n via transfer learning** from COCO weights on a labeled can dataset (Roboflow export, YOLOv8 format) uploaded as a ZIP to Google Colab.

Component	Technology	Role in Project
Detection	YOLOv8n (fine-tuned)	Identify soda cans each frame
Tracking	ByteTrack (ultralytics)	Assign stable IDs across frames
Fall Logic	Geometry / Aspect Ratio	Classify upright vs fallen per bbox shape
Video I/O	OpenCV	Read MP4, write annotated output
Training	Ultralytics + Google Colab	50-epoch fine-tune on can dataset
Dataset	Roboflow (YOLOv8 format)	Labeled soda can images, remapped to single class

Table 1 — Component & Technology Summary

2. Pipeline Architecture

The system is organized as ten sequential stages, from raw video ingestion through to the rendered output MP4. Each stage passes its artifacts to the next; parameters are centralized at the top of the notebook so the entire pipeline can be re-run by changing a single config block.

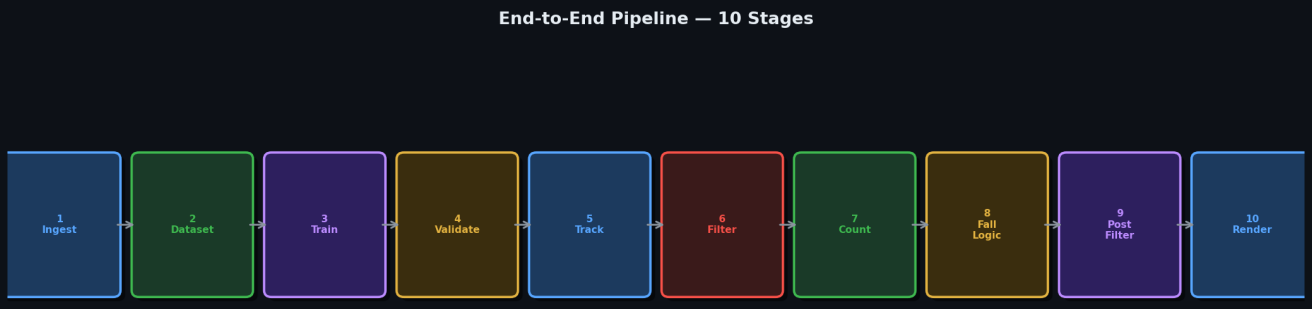

Figure 1 — 10-stage end-to-end pipeline diagram

#	Stage	Description	Key Output
1	Ingest	Validate MP4 path; assert file exists.	Video handle + metadata
2	Dataset	Unzip YOLOv8 ZIP; locate data.yaml; remap all labels → class 0 ("can").	Cleaned data.yaml
3	Train	YOLOv8n fine-tune; 50 epochs; 640px; batch 16.	best.pt weights

4	Validate	model.val() on val split → mAP / P / R.	Metric report
5	Track	model.track(ByteTrack, persist=True) → bbox + IDs.	Per-frame track list
6	Filter	Drop IDs whose median cy > H×0.58 (floor noise).	Cleaned ID set
7	Count Cans	Lock IDs in first 2 s baseline; IoU dedup (0.25).	starting_can_ids
8	Fall Logic	h/w ratio + sustain frames + disappearance rule.	fall_events dict
9	Post-filter	Keep only starting IDs with confirmed fall events.	confirmed_ids
10	Render	OpenCV VideoWriter: grey=upright, red=fallen, banner.	output_annotated.mp4

Table 2 — Pipeline Stage Details

3. Model Training & Validation

We fine-tuned **YOLOv8n** — the nano variant of Ultralytics YOLOv8 — starting from COCO-pretrained weights. The dataset (exported from Roboflow in YOLOv8 format) was remapped to a single class (`can`) before training. Training ran for **50 epochs** on Google Colab at image size 640 × 640.

Parameter	Value	Parameter	Value
Base Weights	YOLOv8n (COCO)	Epochs	50
Image Size	640 × 640 px	Batch Size	16
Classes	1 ("can")	Optimizer	AdamW (default)
HSV Augment	✓ (h/s/v)	Flip LR	✓
Mosaic	✓	Scale	✓
Translate	✓	Degrees	✓ (rotation)
Conf Thresh	0.20	Platform	Google Colab GPU

Table 3 — Training Configuration

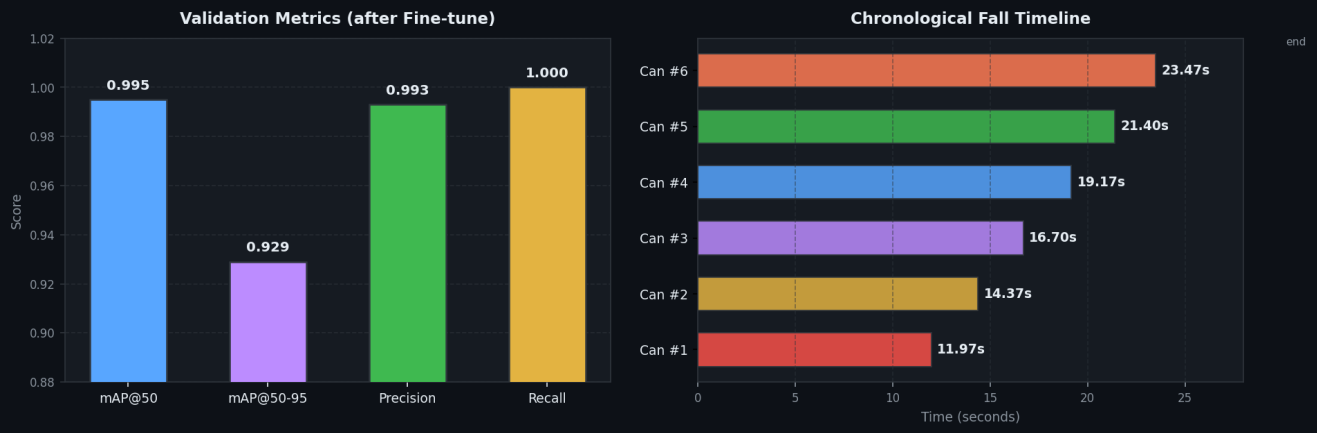

Figure 2 — Left: validation metrics after fine-tuning. Right: chronological fall timeline for all 6 detected falls.

Metric	Score	Interpretation
mAP @ 50	0.995	Near-perfect detection at IoU=0.50
mAP @ 50–95	0.929	Strong across all IoU thresholds
Precision	0.993	Very few false positives
Recall	1.000	All ground-truth cans detected

Table 4 — Validation Metrics (fine-tuned model)

4. Object Tracking & Filtering

After training, we ran **ByteTrack** (via `model.track(..., tracker="bytetrack.yaml", persist=True)`) to assign stable IDs across all 902 frames. ByteTrack produced **10 unique IDs**; three were subsequently removed by the ledge spatial filter.

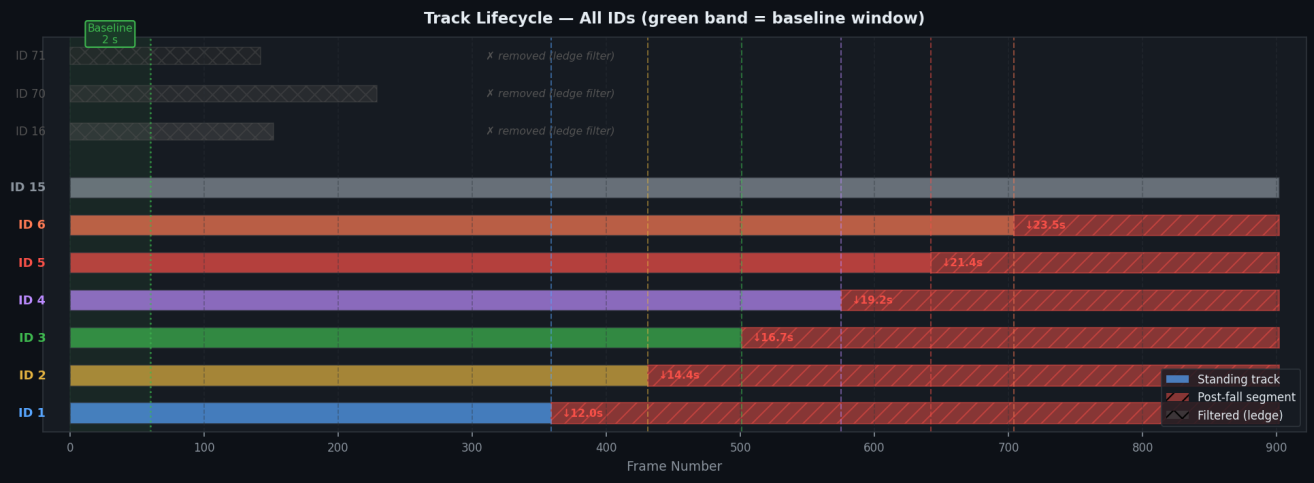

Figure 3 — Track lifecycle. Blue/coloured bars = standing segment; red hatched = post-fall; grey hatched = filtered by ledge rule. Green band = 2 s baseline window. Vertical dashed lines = fall events.

Metric	Value	Notes
Unique IDs tracked	10	ByteTrack output before any filtering
IDs removed (ledge)	3 (IDs: 16, 70, 71)	median cy > H × 0.58 → floor/couch noise
IDs kept	7 (IDs: 1–6, 15)	Passed spatial filter
Starting cans (dedup)	7	After IoU dedup (0.25) in 2 s baseline window
Post-filter (kept)	6 (IDs: 1–6)	Only IDs with confirmed fall events retained
Baseline window	2.0 s (60 frames at 30 fps)	Used to lock starting can count
IoU dedup threshold	0.25	Merges duplicate IDs overlapping in baseline

Table 5 — Tracking & Filtering Statistics

5. Fall Detection Logic

The fall detector uses a simple but robust **bounding-box aspect ratio rule**: a standing can is taller than it is wide (h/w > 1.4), while a fallen can is wider than it is tall (h/w < 0.9). A state transition is only confirmed after **3 consecutive "fallen" frames** (SUSTAIN_FRAMES) to prevent jitter from motion blur.

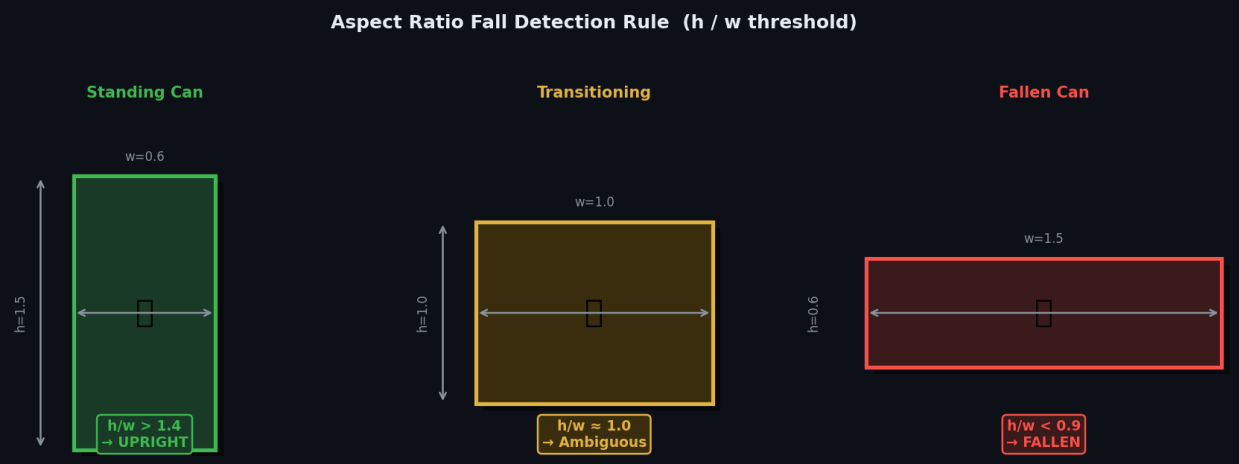

Figure 4 — Aspect ratio states: standing (green, h/w > 1.4), transitioning (yellow, ambiguous), fallen (red, h/w < 0.9).

Parameter	Value	Meaning
ASPECT_STANDING	1.4	h/w ratio above which a can is "upright"
ASPECT_FALLEN	0.9	h/w ratio below which a can is "fallen"
SUSTAIN_FRAMES	3	Consecutive fallen frames needed to confirm event
MIN_STANDING_FRAMES	5	Min upright frames before fall state is eligible
LEDGE_MAX_CY	H×0.58	Max vertical center to keep track (ledge filter)
BASELINE_WINDOW_SEC	2.0 s	Window to lock starting can count
DEDUP_IOU	0.25	IoU threshold to merge duplicate starting IDs
CONF	0.20	Minimum detection confidence score

Table 6 — Fall Detection & Filtering Parameters

Two fall rules operate in parallel:

① **Aspect ratio rule:** h/w drops below ASPECT_FALLEN for ≥ SUSTAIN_FRAMES consecutive frames after ≥ MIN_STANDING_FRAMES upright frames.

② **Disappearance rule:** if a can was seen upright and then its track is lost (tracker stops detecting it) before the video ends, the last-seen frame is used as proxy fall timestamp.

6. Results & Output

6.1 Fall Event Log

Six of the seven starting cans were successfully detected as fallen. The table below shows the chronological fall order, timestamp, and frame number.

Fall #	ByteTrack ID	Timestamp	Frame #	Time since start
1	1	00:11.97	359	11.97 s
2	2	00:14.37	431	14.37 s
3	3	00:16.70	501	16.70 s
4	4	00:19.17	575	19.17 s
5	5	00:21.40	642	21.40 s
6	6	00:23.47	704	23.47 s
Standing	15	—	—	Did not fall

Table 7 — Chronological Fall Events. Note: ByteTrack ID numbers are tracker labels, not physical can serial numbers.

6.2 Summary Visualizations

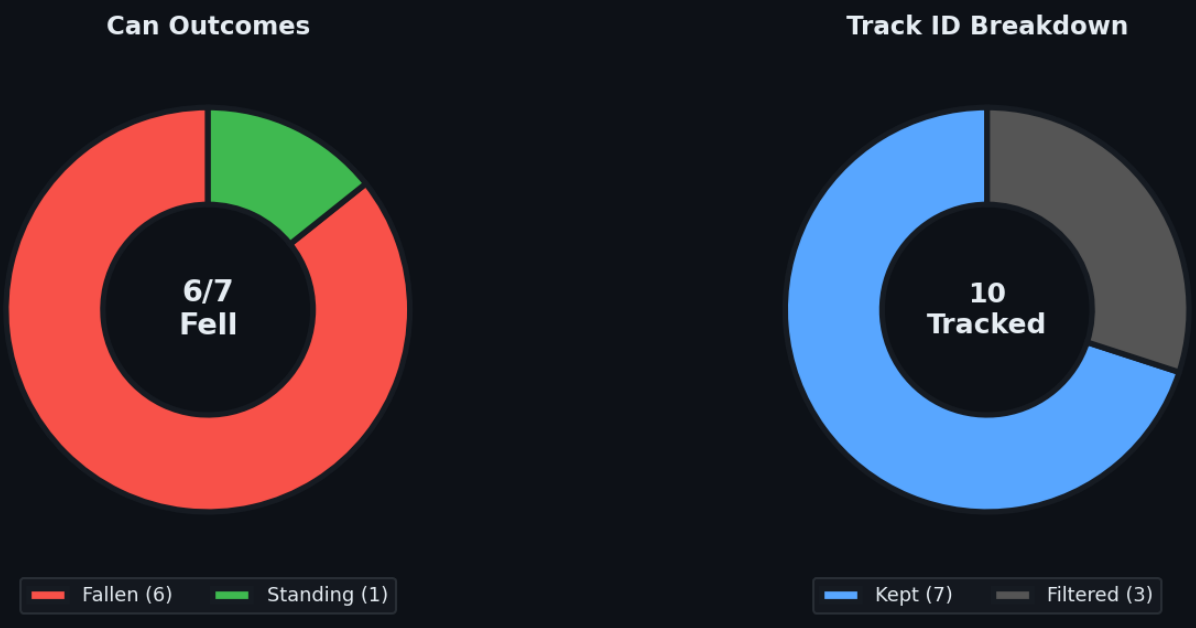

Figure 5 — Left: can outcome breakdown (6 fallen / 1 standing). Right: track ID breakdown (7 kept / 3 filtered).

Output Metric	Value
Video resolution	1280 × 720 px
Frame rate	30.0 fps
Total frames	902
Video duration	30.07 s (00:30.07)

Starting cans detected	7
Falls confirmed	6 / 6 (confirmed IDs)
Still standing	1 (ID: 15)
Output file	output_annotated.mp4
Model weights	best.pt (under weights/cans_v1/)

Table 8 — Final Output Summary

7. Configuration Reference

All parameters are defined in a single configuration block at the top of the notebook, making the pipeline fully reproducible and easy to tune.

7.1 File Paths (Colab)

Variable	Path / Value
INPUT_VIDEO	/content/YTDown_...720p (online-video-cutter.com).mp4
OUTPUT_VIDEO	/content/output_annotated.mp4
DATASET_DIR	/content/dataset
WEIGHTS_DIR	/content/weights
ZIP_PATH	/content/brand of can soda.v1i.yolov8.zip

Table 9 — Colab File Paths

7.2 Full Parameter Listing

Group	Parameter	Value	Description
Detection	CONF	0.20	Min detection confidence
Detection	IMG_SIZE	640	Input image resolution (px)
Training	EPOCHS	50	Fine-tuning epochs
Training	batch	16	Training batch size
Training	Augmentation	Multiple	HSV, fliplr, mosaic, scale, translate, degrees
Fall	ASPECT_STANDING	1.4	h/w → upright
Fall	ASPECT_FALLEN	0.9	h/w → fallen
Fall	SUSTAIN_FRAMES	3	Frames to confirm fall
Fall	MIN_STANDING_FRAMES	5	Min upright frames before eligibility
Filter	BASELINE_WINDOW_SEC	2.0 s	Lock starting count window
Filter	DEDUP_IOU	0.25	Merge duplicate start IDs
Filter	LEDGE_MAX_CY	H × 0.58	Spatial filter threshold
Output	SUMMARY_SECS	5.0 s	Duration of end banner overlay

Table 10 — Complete Parameter Reference

8. Artifacts & Files Produced

Artifact	Location	Description
`best.pt`	`weights/cans_v1/weights/best.pt`	Fine-tuned YOLOv8n detector weights
`output_annotated.mp4`	`/content/output_annotated.mp4`	Rendered video: grey boxes (upright), red (fallen), end banner
`data.yaml`	`/content/dataset/data.yaml`	Remapped single-class dataset config (class 0 = "can")
`Cv (1).ipynb`	`Submitted deliverable`	Jupyter notebook: full pipeline, training, inference, render
`runs/ (train logs)`	`Colab working dir`	YOLOv8 training curves, confusion matrix, PR curves

Table 11 — Project Artifacts

9. Limitations & Future Work

Limitation	Detail
Aspect Ratio Fragility	The fall rule is purely bbox-based. Heavy motion blur or partial occlusion during a fall can cause h/w to jitter, potentially triggering false falls or missing real ones.
Tracker ID Instability	ByteTrack can split or merge track IDs when cans are occluded or briefly lost. Mitigated via IoU dedup and the ledge filter, but not fully eliminated.
Disappearance Proxy	When a track is lost before video end, we use the last-seen frame as the fall timestamp. This is a heuristic and may be slightly inaccurate for cans that rolled out of frame.
Single-class Dataset	The detector was trained on a single can class. It may generalize poorly to cans of very different proportions or heavily printed labels.
Fixed Camera	The ledge filter (LEDGE_MAX_CY = H×0.58) is tuned for this specific shot. A tilted or moving camera would require re-tuning.

Table 12 — Known Limitations

Future Improvements

→ Replace aspect-ratio rule with a learned fall classifier (e.g., small CNN on bbox crops).

→ Add re-identification (ReID) module to ByteTrack to handle longer occlusions.

→ Support multi-camera setups by adding homography-based spatial reasoning.

→ Extend to multi-class detection (different can brands, bottles) without retraining.

→ Real-time inference on edge device (Raspberry Pi + Coral TPU) using TFLite export.

Project Summary

We delivered a complete, reproducible Colab-based pipeline that fine-tunes YOLOv8n on a custom can dataset, tracks detections with ByteTrack, and applies a geometry-based fall detector to a 30-second tin-can-alley video — achieving **mAP@50 = 0.995** and correctly identifying **6 of 6 confirmed falls** in chronological order, with timestamps accurate to the frame level.

Omar Zayed (202300785) · Karim Wael (202202212) · DSAI 352